

Tabular Database Systems

⑧

SQL: Grouping + Aggregation and Functional Dependencies

Summer 2025

Torsten Grust
Universität Tübingen, Germany

1 | SQL: Grouping (Clause **GROUP BY**)

- Up to now, SQL queries operate *row-by-row*:
 - FROM** t binds *individual rows* of t to row variables,
 - WHERE** p drops row var bindings that do not satisfy p ,
 - SELECT** $e, \dots, e$ evaluates $e, \dots, e$ for each row var binding.

Clause **GROUP BY** $e_1, \dots, e_n$ identifies **subtables (or: groups) of rows** that **yield the same value for expressions (criteria) $e_1, \dots, e_n$** :

vehicles					groups
vehicle	kind	seats	wheels?	driver	
	bike	1	true	p_2	true
	cabrio	2	true	p_4	
	SUV	3	true	p_4	
	car	5	true	p_4	false
	bus	42	true	$\square$	
	bus	7	true	$\square$	
	tank	$\square$	false	p_3	$\square$

SELECT ...
 FROM vehicles AS v
 GROUP BY v.seats < 5
 grouping criterion

∀ rows v in this group:
 v.seats < 5 ≡ false

SQL: Grouping Requires Aggregation

💡 After grouping, use **aggregation** to represent group contents:

1 📊 **vehicles** (grouped)

vehicle	...	seats	...	driver
🏍️		1		p_2
🚗	...	2	...	p_4
🚙		3		p_4
🚗		5		p_4
🚌	...	42	...	□
🚐		7		□
🚚	...	□	...	p_3

2 📊 **vehicles** (grouped + aggregated)

seats<5	count(*)	max(seats)	list(driver)
true	3	3	$[p_2, p_4, p_4]$
false	3	42	$[p_4, \square, \square]$
□	1	□	$[p_3]$

- Tables are in 1NF. Cells cannot contain groups/subtables. Thus:
 - SQL never provides access to the grouped table **1** itself.
 - **Aggregation after grouping **2** is mandatory.**

GROUP BY Reduces Granularity (Row-by-Row to Group-by-Group)

- The **FROM** ❶ and **WHERE** ❷ clauses operate row-by-row.
- After **GROUP BY** ❸, the **SELECT** ❹ clause operates group-by-group and yields **one row per group**:

```

SELECT   $expr_1, \dots, expr_n,$ 
         $agg, \dots, agg$ 
FROM     $t$ 
WHERE    $p$ 
GROUP BY  $expr_1, \dots, expr_n$ 

```

❸ } g
 ❹ }
 ❶ $n (= |t|)$
 ❷ $m (\leq n)$
 ❸ $g (\leq m)$
 # rows

o1 #048

Non-aggregate $expr_i$ in **SELECT** is OK only if (the DBMS can statically deduce that) $expr_i$ is *constant within each group*.

- Grouping Quiz:** What can you infer if you observe the following?
 Ⓐ $g = 1$. Ⓑ $g = m$. Ⓒ $g = 0$.
 Ⓓ Adding a new $expr_j$ to the **GROUP BY** clause does not change g .

Controlling Granularity: Derived Grouping Criteria

```
SELECT t.c,agg,...,agg
FROM   t
GROUP BY t.c -- criterion c leads to too many groups... ˘\_(\ツ)\_/˘
```

 #049

1. **Division.** Group numeric c into buckets of **equal width** $1/N$:
 - If c is integral (int): **GROUP BY** $c // N$
 - Otherwise: **GROUP BY** $\text{round}(c / N)$
2. **Modulus.** Put integral c into one of N buckets (**order lost**):
 - **GROUP BY** $c \% N$
 - If $N = 2^n$: **GROUP BY** $c \& (1 \ll n) - 1$ (based on n low bits)
3. **Grading.**¹ Place ordered c into buckets with **given borders** b_i :



¹ *Grading* is derived from APL's $\uparrow$ (“grade up”), `list_grade_up()` in DuckDB: `list_grade_up([30,10,20]) = [2,3,1]` are the list indices that would rearrange `[30,10,20]` in ascending order.

A Grouping Playground: The New York City Taxi Database

The *Yellow Cab* database² published by the NYC Taxi & Limousine Commission (TLC) provides a fun playground for **GROUP BY** queries.

A DuckDB database file holds the dataset for year 2024:

 #051

 rides	main table, one row per taxi trip
 zones	division of New York City into 265 zones (“boroughs”), rides start/stop here
 central_park_weather	temperature/wind/precipitation data for Central Park, one row per day

Find plain + embedded SQL queries over this data in

 #050  #052

² Published monthly on the [TLC Trip Record Data page](#) , (formerly in CSV, since 2022 in Parquet format).

2 : Group Preservation

```
-- How many peeps can each driver give a lift?
SELECT p.pid AS driver, ,
       max(v.seats) - 1 AS "can lift"
FROM   vehicles AS v NATURAL JOIN peeps AS p
GROUP BY p.pid, ;
```

 #053

vehicles ⋈ peeps

vehicle	kind	seats	wheels?	pid	pic	name	born
	bike	1	true	2		Bert	1968
	tank	0	false	3		Drew	0
	car	5	true	4		Alex	2002
	SUV	3	true	4		Alex	2002
	cabrio	2	true	4		Alex	2002

- Adding `p.name` (`pic`, `born`) to `GROUP BY`  preserves groups:
rows that agree on `pid`, also agree on `name`: `pid` → `name`.

Functional Dependencies (FDs)

Given table $t(c_1, \dots, x, \dots, y, \dots, c_n)$, the **functional dependency (FD)** $x \rightarrow y$ (“column x determines column y ”) holds in t iff

$$\forall \text{ rows } r_1, r_2 \in t: r_1.x = r_2.x \Rightarrow r_1.y = r_2.y.$$

• Notes on FDs:

- FD $x \rightarrow y$ indicates that t **embeds a lookup table** $t^f(\underline{x}, y)$ defining a function $f(x) = y$.³ Embedded table t^f has key x .
- Generalization: $x_1 \ x_2 \ \dots \ x_n \rightarrow y$ defines an n -ary function.
- If $x \rightarrow y$ and $y \rightarrow z$, then $x \rightarrow z$ (transitivity, like $g \circ f$).
- Shorthand notation: $x \rightarrow y$ and $x \rightarrow z \equiv x \rightarrow y \ z$.
- A key  k for table $t(\underline{k}, c_1, \dots, c_n)$ is like a special, powerful FD: $k \rightarrow c_1 \ \dots \ c_n$ (key k determines all columns).
- Trivial FD (holds for any column x in any table t): $x \rightarrow x$.
- $x \rightarrow y$ does *not* generally imply $y \rightarrow x$.

 #053

³ Example: For the FD $\text{pid} \rightarrow \text{pic}$ in `vehicles` $\bowtie$ `peeps`, we have $f(2) = \text{🐶}$, $f(3) = \text{🐱}$, $f(4) = \text{🐱}$ (3x).

SQL: HAVING (and a Test for Functional Dependencies)

- SQL clause **HAVING** is evaluated *after* groups have been formed. Predicate q thus may use aggregates to **filter entire groups**:

SELECT	$expr, \dots, expr,$	
	$agg, \dots, agg$	
FROM	t	
WHERE	p	
GROUP BY	$expr, \dots, expr$	
HAVING	q	

- A **HAVING**-based test: Does FD $x \rightarrow y$ hold in table t ?

 #054

SELECT DISTINCT	$'x \not\rightarrow y'$	AS	"FD violated?"	
FROM	t			
GROUP BY	x			
HAVING	$\text{count}(\text{DISTINCT } y) > 1$			

3 : Apps Define FDs ...

 trips

trip	weekday	driver	vehicle	from	to	dist	via	hop#
t_1	Mon	4		Alton	Corby	150	Alton	1
t_1	Mon	4		Alton	Corby	150	Luton	2
t_1	Mon	4		Alton	Corby	150	Corby	3
t_2	Tue	2		Derby	Eaton	17	Derby	1
t_2	Tue	2		Derby	Eaton	17	Eaton	2
t_3	Wed	2		Derby	Crich	23	Derby	1
t_3	Wed	2		Derby	Crich	23	Crich	2
t_4	Thu	4		Alton	Corby	150	Alton	1
t_4	Thu	4		Alton	Corby	150	Luton	2
t_4	Thu	4		Alton	Corby	150	Corby	3

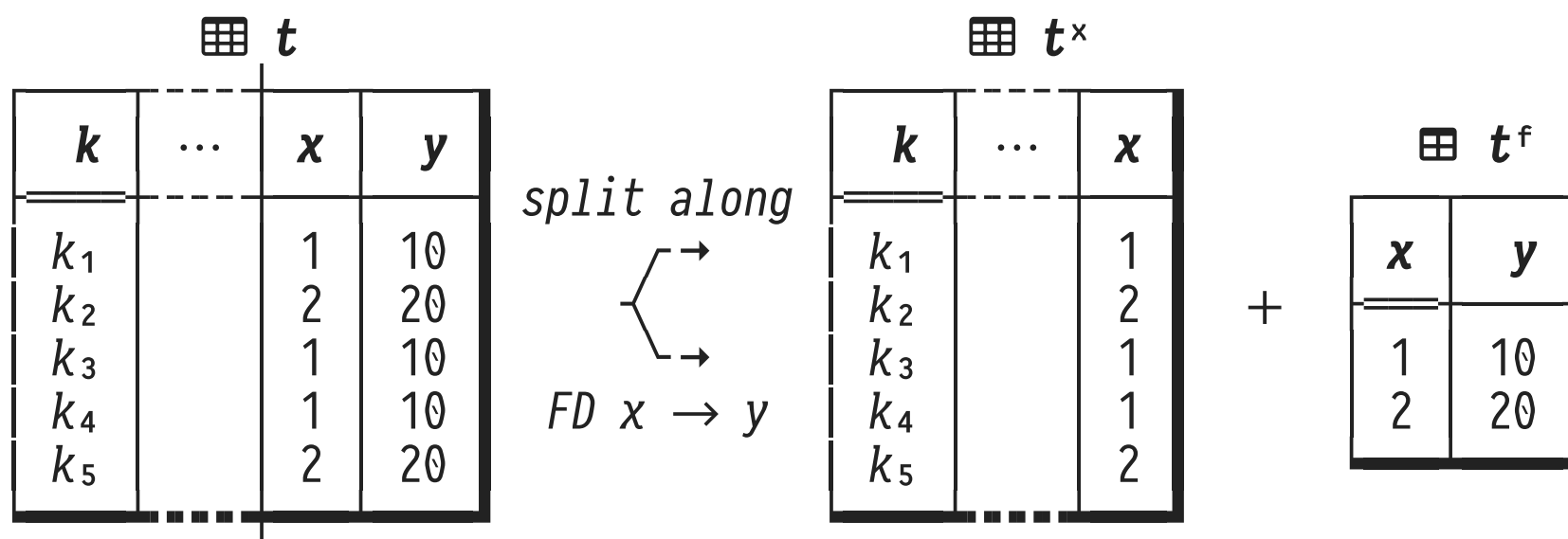
Schedule of multi-hop road trips taken (with drivers and vehicles in use).

- Here are some FDs that hold in table **trips**. Verbalize what they mean in a DB application that manages road trips:

- Ⓐ vehicle $\rightarrow$ driver Ⓑ trip $\rightarrow$ weekday from to Ⓒ weekday $\rightarrow$ driver
 Ⓓ from to $\rightarrow$ dist driver Ⓔ from to hop# $\rightarrow$ via vehicle

... FDs Suggest Good Table Designs

1. The only FDs actually enforced by DBMSs are primary keys : recall the **PRIMARY KEY** constraint.
2. Non- FDs (embedded lookup tables) indicate data redundancy.



 #055

- After the **split** $\boxtimes \rightarrow \boxtimes + \boxtimes$ along FD $x \rightarrow y$:⁴
 - x is key in t^f : the DBMS can now enforce the FD $x \rightarrow y$. 
 - **No redundancy** in t^f . Function $f(x) = y$ is now explicit. 

⁴ Advise: When splitting along FD $x \rightarrow y$, move *all* columns functionally determined by x into t^f (including transitively determined columns). If t^f contains non- FDs, split t^f again.

Splits Can Be Undone (Virtually)

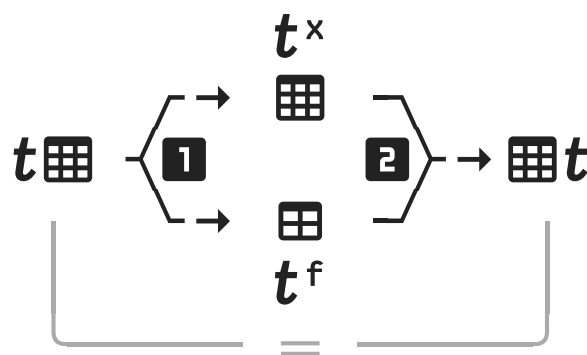
FD splits improve constraint checking and reduce redundancy. But users or apps may require/expect the original non-splitted table...

- An inner join $\bowtie$ can **faithfully reconstruct** the original t .
Table state is preserved (rows are neither lost nor added):⁵

FD split **1**

2 Inner join $\bowtie$

 #056



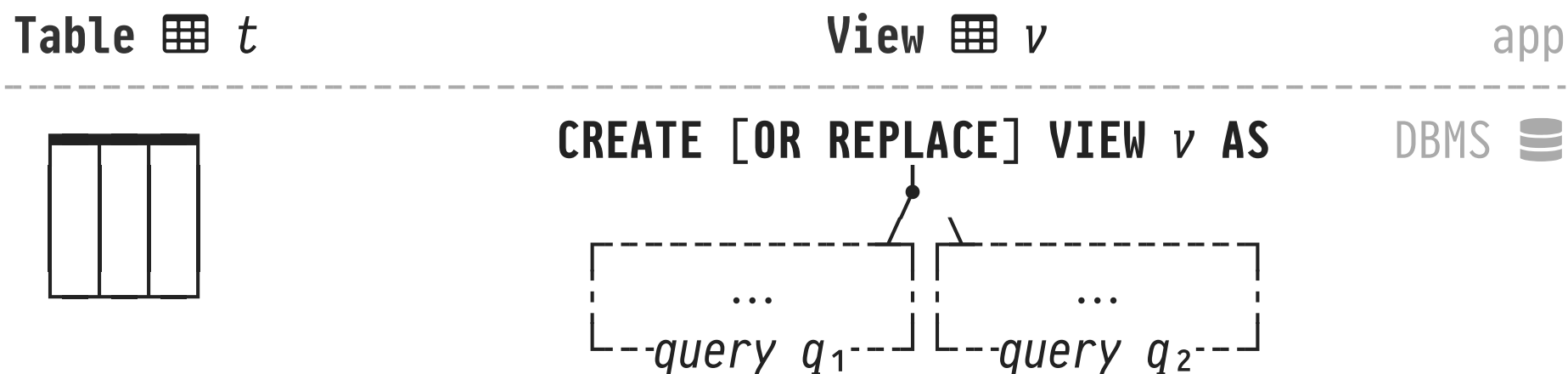
- A **table view** can render this reconstruction of t transparent.

⁵ FD splits are **lossless splits**.  But: arbitrary vertical table splits may lose information ($t \neq t_1 + t_2 + t_3$).

4 : Views (Logical Data Independence)

A SQL **view** v represents a *virtual* table whose schema and state is **defined by a query** q (instead of extensionally by a bag of rows).

- Views provide **logical data independence**. The actual data sources read by q remain hidden (q may change $\nrightarrow \leftrightarrow \nleftarrow$):



- Whenever v is referenced by a query, the DBMS re-evaluates q .
- Views are read-only and cannot be updated.⁶

⁶ In general, it is ambiguous how an “update” of v should affect the underlying source tables read by q .

The End.

Next semester, please join us for
Design and Implementation of Database Internals (DiDi⁷)
Advanced SQL

⁷ Dissecting the Duck's innards 🦆